Specyfikacja relacji SQL DDL i wdrożenie bazy danych

System zarządzania czasem pracy pracowników

Podstawa opracowania	Technologia	Zakres
„Bazy projekt-2.pdf" oraz „bazy projekt (1).sql"	PostgreSQL 14 lub nowszy	24 relacje, model logiczny w III postaci normalnej

6. Opracowanie specyfikacji relacji w formie SQL DDL

6.1. Założenia implementacyjne

Podstawą implementacji jest model logiczny systemu ewidencji czasu pracy opisany w załączonym projekcie (rozdziały 3–4). Obejmuje on 21 encji modelu konceptualnego oraz 3 relacje pośrednie odwzorowujące związki wiele-do-wielu. Przyjęto angielskie nazwy tabel i kolumn z dołączonego skryptu, aby zachować zgodność z istniejącym diagramem relacji.

Fizyczny model utworzono w schemacie work_time. Klucze identyfikacyjne (w tym assignment_id tabeli Assignment) otrzymują automatycznie generowane wartości. Trzy proste relacje łącznikowe mają klucze złożone. Daty zapisano jako DATE, przedziały godzinowe jako TIMESTAMP, a wymiar etatu jako NUMERIC(3,2).

Decyzja projektowa: W tabeli Assignment pozostawiono assignment_id jako samodzielny klucz główny, zgodnie z dołączonym plikiem SQL. Para (shift_id, employee_id) została objęta ograniczeniem UNIQUE, aby uniemożliwić podwójne przypisanie pracownika do jednej zmiany. Rozwiązanie świadomie różni się od wersji modelu logicznego w PDF, gdzie ta para stanowiła klucz główny. Unikalność pary zachowuje jej rolę klucza kandydującego i nie narusza przyjętej III postaci normalnej.

6.2. Zestawienie relacji, kluczy głównych i powiązań

Relacja SQL	Klucz główny (PK)	Klucze obce (FK)
Branch	branch_id	—
Department	department_id	branch_id → Branch
Team	team_id	department_id → Department
Position	position_id	—
Zone	zone_id	branch_id → Branch
Employee	employee_id	team_id → Team; manager_id → Employee
Account	account_id	employee_id → Employee (UNIQUE)
Role	role_id	—
Permission	permission_id	—
AccountRole	account_id + role_id	Account; Role
RolePermission	role_id + permission_id	Role; Permission
ContractType	contract_type_id	—
Contract	contract_id	Employee; ContractType; Position
Annex	annex_id	Contract; Position
Schedule	schedule_id	department_id → Department

Relacja SQL	Klucz główny (PK)	Klucze obce (FK)
Shift	shift_id	schedule_id → Schedule
Assignment	assignment_id	shift_id → Shift; employee_id → Employee; zone_id → Zone
Availability	availability_id	Employee; Account
AbsenceType	absence_type_id	—
Absence	absence_id	Employee; AbsenceType; Account
Registration	registration_id	Employee; Shift (opc.); Account
DayOff	day_off_id	—
BranchDayOff	branch_id + day_off_id	Branch; DayOff
Audit	audit_id	account_id → Account (opc.)

Pełne definicje atrybutów, typów danych i wszystkich ograniczeń znajdują się w pliku 01_schema_postgresql.sql; zestawienie powyżej prezentuje strukturę powiązań relacyjnych.

6.3. Wykorzystane instrukcje SQL DDL

Instrukcja / mechanizm	Zastosowanie w projekcie
CREATE SCHEMA	Wydzielenie przestrzeni work_time na obiekty aplikacji.
CREATE TABLE	Definicja 24 tabel, kolumn, typów danych oraz wartości domyślnych.
PRIMARY KEY	Jednoznaczna identyfikacja rekordów: Assignment ma własny assignment_id; relacje AccountRole, RolePermission i BranchDayOff mają klucze złożone.
FOREIGN KEY / REFERENCES	Kontrola integralności referencyjnej, np. pracownik → zespół, zmiana → grafik.
NOT NULL, UNIQUE	Ograniczenie braków i duplikatów: login, konto pracownika, numer aneksu oraz para (shift_id, employee_id) w Assignment.
CHECK	Walidacja dat, pojemności stref, wymiaru etatu oraz dopuszczalnych statusów.
DEFAULT, IDENTITY	Wartości domyślne i automatycznie generowane identyfikatory, w tym assignment_id.
CREATE INDEX	Przyspieszenie łączeń i filtrowania danych wg kluczy obcych i dat.
CREATE UNIQUE INDEX ... WHERE	Blokowanie dwóch wersji opublikowanych grafiku dla identycznego działu i okresu.
BEGIN / COMMIT	Transakcyjne utworzenie schematu: błąd wycofuje zmiany w DDL.

6.4. Przykładowe fragmenty specyfikacji

Przydzielenie pracownika do zmiany i jednej strefy realizuje relacja Assignment. Każdy przydział posiada własny assignment_id, a para (shift_id, employee_id) jest dodatkowo unikalna:

```
CREATE TABLE "Assignment" (
  "assignment_id" INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "shift_id" INTEGER NOT NULL REFERENCES "Shift" ("shift_id"),
  "employee_id" INTEGER NOT NULL REFERENCES "Employee" ("employee_id"),
  "zone_id" INTEGER NOT NULL REFERENCES "Zone" ("zone_id"),
  CONSTRAINT "uq_assignment_shift_employee" UNIQUE ("shift_id", "employee_id")
);
```

Przykład warunku poprawności danych — pojemność strefy musi być dodatnia:

```
CONSTRAINT "ck_zone_capacity" CHECK ("capacity" > 0)
```

Przykład indeksu częściowego dopuszczającego co najwyżej jedną opublikowaną wersję grafiku dla wskazanego działu i dokładnie takiego samego okresu:

```
CREATE UNIQUE INDEX "uq_schedule_one_published_version"
  ON "Schedule" ("department_id", "period_from", "period_to")
  WHERE "status" = 'published';
```

6.5. Spójność z modelem logicznym

Schemat zachowuje oddzielenie pracowników, struktur organizacyjnych, umów, aneksów, grafików, przydziałów, rejestracji i uprawnień. Role i uprawnienia łączą tabele AccountRole i RolePermission, a dni wolne z oddziałami — BranchDayOff. Dzięki takiemu rozdzieleniu ogranicza się powtarzanie danych, zgodnie z założeniami III postaci normalnej opisanej w projekcie.

Wybrane ograniczenia techniczne doprecyzowują model: 0 < employment_fraction ≤ 1; pojemność strefy > 0; data końcowa nie może być wcześniejsza niż początkowa; numery aneksów są unikalne w obrębie umowy. Są to założenia implementacyjne i można je zmienić, jeżeli polityka przedsiębiorstwa wymaga innych zasad.

7. Wdrożenie projektu w systemie PostgreSQL

7.1. Środowisko docelowe

Jako relacyjny system zarządzania bazą danych wybrano PostgreSQL (zalecana wersja 14 lub nowsza). Baza nosi nazwę ewidencja_czasu_pracy, a obiekty aplikacji znajdują się w schemacie work_time. Skrypt można uruchomić z poziomu narzędzia psql lub pgAdmin (Query Tool). Skrypt bazowy nie wymaga dodatkowych rozszerzeń PostgreSQL.

7.2. Instrukcja wdrożenia — konsola psql

Po instalacji i uruchomieniu serwera PostgreSQL należy zalogować się jako użytkownik uprawniony do tworzenia baz danych. W konsoli systemowej wykonać:

```
createdb -U postgres -E UTF8 ewidencja_czasu_pracy
```

Następnie w katalogu, w którym znajdują się pobrane skrypty:

```
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 -f 01_schema_postgresql.sql
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 -f 02_weryfikacja_i_testy.sql
```

W systemie Windows te same pliki można wykonać przez terminal psql (z odpowiednią ścieżką do pliku) albo wkleić do edytora zapytań w pgAdmin. Drugi skrypt zawiera polecenia specyficzne dla psql (np. \echo), dlatego w pgAdmin weryfikację należy uruchamiać jako osobne zapytania SQL lub skorzystać z psql.

Uwaga eksploatacyjna: Skrypt 01 jest przeznaczony do utworzenia nowego schematu i tabel w pustej bazie. Nie należy uruchamiać go drugi raz na istniejącym schemacie bez przygotowania migracji. Przed zmianami w używanym środowisku produkcyjnym należy wykonać kopię zapasową.

7.3. Weryfikacja utworzonych obiektów

Poniższe zapytanie powinno zwrócić 24, ponieważ tyle relacji przewiduje model logiczny:

```
SELECT COUNT(*) AS liczba_tabel
FROM information_schema.tables
WHERE table_schema = 'work_time' AND table_type = 'BASE TABLE';
```

Dodatkowo w psql można użyć poleceń:

```
\dn
\dt work_time.*
\d work_time."Assignment"
```

Skrypt 02_weryfikacja_i_testy.sql sprawdza liczbę tabel, rodzaje zdefiniowanych ograniczeń, pojedynczy klucz główny assignment_id, automatyczne nadawanie identyfikatora, unikalność pary (shift_id, employee_id) oraz podstawową poprawność kluczy obcych. Tworzy niewielki zestaw danych testowych w transakcji, sprawdza odrzucanie niepoprawnych wpisów i na koniec wycofuje wszystkie dane testowe poleceniem ROLLBACK.

7.4. Przypadki testowe i oczekiwane wyniki

Test	Oczekiwany wynik
Liczba tabel w work_time	24
Klucz główny Assignment	Jedna kolumna: assignment_id (IDENTITY)
Ponowne dodanie tego samego pracownika do tej samej zmiany	Odrzucenie INSERT — naruszenie UNIQUE (shift_id, employee_id)
Strefa z capacity = 0	Odrzucenie INSERT — naruszenie CHECK
Przydział do nieistniejącego pracownika	Odrzucenie INSERT — naruszenie FOREIGN KEY
Poprawny pracownik, zmiana i strefa	Wynik zapytania JOIN zawierający dane przydziału
Stan danych po zakończeniu testów	ROLLBACK usuwa testowe rekordy, pozostawia strukturę
Automatyczna generacja identyfikatora przydziału	assignment_id jest automatycznie nadawany podczas INSERT

7.5. Granice implementacji DDL i dalsze elementy systemu

SQL DDL wdraża strukturę i proste ograniczenia integralności, ale nie zapewnia samodzielnie wszystkich procesów opisanych w wymaganiach F01–F36. Do pełnej realizacji funkcjonalności potrzebne są dodatkowe zapytania, procedury lub triggery PostgreSQL oraz warstwa aplikacyjna. Dotyczy to m.in. wykrywania cykli w hierarchii przełożonych (F12), kolizji przydziałów i pojemności stref (F20), pełnej kontroli publikacji i nakładania się okresów grafików (F21–F22), sprzecznych deklaracji dostępności (F24), kontroli sekwencji zdarzeń czasu pracy (F30–F32) oraz automatycznego i bezpiecznego audytu (F36).

Indeks częściowy dla Schedule chroni tylko przed równoczesnym opublikowaniem dwóch wersji dla identycznego działu i identycznych granic okresu. Nie blokuje samodzielnie częściowo nakładających się okresów; taki wymóg wymaga osobnego mechanizmu. Skrypt nie zawiera też gotowych kont użytkowników ani uprawnień dostępu do serwera — te należy skonfigurować zgodnie z polityką bezpieczeństwa.

7.6. Status przygotowania i wnioski

Przygotowano plik DDL przeznaczony do uruchomienia w PostgreSQL oraz osobny skrypt weryfikacyjny. Wykonano kontrolę liczby relacji oraz podstawowych więzów integralności na lokalnej, zaadaptowanej wersji modelu. Nie przeprowadzono rzeczywistego uruchomienia skryptów na instancji PostgreSQL w

środowisku przygotowania tego opracowania; dlatego przedstawione wyniki dla PostgreSQL stanowią oczekiwane wyniki testów, a nie protokół już wykonanego wdrożenia.

Po wykonaniu skryptów na docelowym serwerze, zapisaniu komunikatów testowych i potwierdzeniu liczby 24 tabel można udokumentować zakończenie etapu fizycznego wdrożenia schematu. Pełny system operacyjny będzie wymagał dodatkowej implementacji reguł biznesowych i interfejsu użytkownika.

8. Więzy integralności i operacje kaskadowe

System zarządzania czasem pracy — implementacja w PostgreSQL (schemat work_time). Opracowanie oparto na modelu 24 relacji z projektu oraz na aktualnym skrypcie DDL z kluczem Assignment.assignment_id. Wybrane akcje referencyjne przedstawione w tym rozdziale są rozszerzeniem wcześniejszego skryptu, a nie zapisami już istniejącymi w pierwotnym modelu.

8.1. Rodzaje więzów integralności

Rodzaj	Realizacja SQL	Zastosowanie w projekcie
Integralność encji	PRIMARY KEY, NOT NULL	Unikalne identyfikatory 24 tabel; Assignment.assignment_id jest kluczem głównym.
Integralność referencyjna	FOREIGN KEY, REFERENCES	Powiązania Employee–Team, Contract–Employee, Assignment–Shift/Employee/Zone itd.
Integralność unikalności	UNIQUE, unikalny indeks	Login i employee_id w Account; (shift_id, employee_id) w Assignment; jedna opublikowana wersja Schedule dla tego samego okresu.
Integralność dziedzinowa	CHECK, NOT NULL, typy danych	Dodatnia pojemność Zone, etat (0;1], okresy, dozwolone statusy i typy zdarzeń.

Więzy są egzekwowane przez silnik PostgreSQL podczas wstawiania i aktualizacji danych; więzy referencyjne kontrolują także skutki usuwania rekordów powiązanych. Przykładowo próba przypisania pracownika, którego employee_id nie istnieje, kończy się błędem klucza obcego.

```
CREATE TABLE "Assignment" (
   "assignment_id" INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   "shift_id" INTEGER NOT NULL REFERENCES "Shift" ("shift_id"),
   "employee_id" INTEGER NOT NULL REFERENCES "Employee" ("employee_id")
      ON DELETE RESTRICT,
   "zone_id" INTEGER NOT NULL REFERENCES "Zone" ("zone_id"),
   CONSTRAINT "uq_assignment_shift_employee"
      UNIQUE ("shift_id", "employee_id")
);
```

assignment_id ułatwia wskazanie konkretnego przydziału, natomiast UNIQUE(shift_id, employee_id) nadal gwarantuje, że w obrębie jednej zmiany pracownik wystąpi najwyżej raz, a więc będzie miał najwyżej jedną strefę pracy.

8.2. Dostępne działania referencyjne

PostgreSQL udostępnia pięć działań przy ON DELETE i ON UPDATE. Klauzula ON DELETE określa skutek usunięcia rekordu nadrzędnego; ON UPDATE — zmiany wartości jego klucza. Domyślna reguła dla obydwu zdarzeń to NO ACTION.

Działanie	Znaczenie
NO ACTION	Operacja jest niedozwolona, jeśli przy sprawdzeniu klucza obcego pozostaje błędne odwołanie. W odróżnieniu od RESTRICT może podlegać odroczonej weryfikacji, jeżeli więź zdefiniowano jako odraczalną.
RESTRICT	Blokuje usunięcie/zmianę klucza nadrzędnego, gdy istnieją

Działanie	Znaczenie
	zależne rekordy; nie pozwala odroczyć tego sprawdzenia.
CASCADE	ON DELETE: usuwa zależne rekordy; ON UPDATE: przepisuje nową wartość klucza nadrzędnego do FK.
SET NULL	Ustawia odpowiednie kolumny FK rekordu zależnego na NULL; wymaga, by kolumna dopuszczała NULL.
SET DEFAULT	Ustawia wartość domyślną FK; musi ona spełniać ograniczenie FK albo być dopuszczalnym NULL.

W projekcie zastosowano jawnie CASCADE, SET NULL i RESTRICT. W pozostałych relacjach pozostawiono domyślne NO ACTION. SET DEFAULT omówiono teoretycznie, lecz nie wdrożono: sztuczny „domyślny" pracownik, oddział czy zmiana nie odpowiadałby semantyce danych.

8.3. Działania kaskadowe zastosowane w projekcie

Klucz obcy	ON DELETE	ON UPDATE	Uzasadnienie
AccountRole.account_id → Account	CASCADE	CASCADE	Usuwane są tylko przypisania ról do usuwanego konta.
AccountRole.role_id → Role	CASCADE	CASCADE	Usunięcie roli usuwa jej przypisania do kont.
RolePermission.role_id → Role	CASCADE	CASCADE	Usunięcie roli usuwa związki roli z uprawnieniami.
RolePermission.permission_id → Permission	CASCADE	CASCADE	Usunięcie uprawnienia usuwa przypisania do ról.
BranchDayOff.branch_id → Branch	CASCADE	CASCADE	Usunięcie oddziału usuwa tylko wiązania z dniami wolnymi (jeśli inne FK pozwolą).
BranchDayOff.day_off_id → DayOff	CASCADE	CASCADE	Usunięcie dnia wolnego usuwa tylko powiązania z oddziałami.
Employee.manager_id → Employee	SET NULL	CASCADE	Zerowanie opcjonalnego odwołania do usuniętego przełożonego.
Contract.employee_id → Employee	RESTRICT	NO ACTION	Blokada usunięcia pracownika z zachowaną umową.
Assignment.employee_id → Employee	RESTRICT	NO ACTION	Blokada usunięcia pracownika z przydziałem do zmiany.
Registration.employee_id → Employee	RESTRICT	NO ACTION	Blokada usunięcia pracownika z rejestracjami czasu pracy.

W tabelach pośrednich kaskada usuwa jedynie skojarzenia, nie obiekty docelowe. Przykładowo usunięcie roli usuwa wiersze AccountRole i RolePermission, lecz nie usuwa pracowników, kont ani samych uprawnień. Z kolei dla Contract, Assignment i Registration chronimy dane historyczne, zamiast usuwać je automatycznie.

```
ALTER TABLE "RolePermission"
  ADD CONSTRAINT "fk_rolepermission_role"
  FOREIGN KEY ("role_id") REFERENCES "Role" ("role_id")
  ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE "Employee"
  ADD CONSTRAINT "fk_employee_manager"
  FOREIGN KEY ("manager_id") REFERENCES "Employee" ("employee_id")
  ON DELETE SET NULL ON UPDATE CASCADE;
```

Są to fragmenty ilustrujące nową definicję więzów; podczas migracji wcześniejsze klucze obce trzeba najpierw usunąć, a następnie dodać ponownie. Gotowy, pełny zestaw poleceń znajduje się w pliku 03_migracja_wiezow_integralnosci.sql.

Uwaga praktyczna: zwykłe usuwanie pracowników nie jest zalecane. Projekt przewiduje oznaczanie ich jako archiwalnych (Employee.archived). SET NULL dla manager_id może zadziałać tylko wtedy, gdy inne klucze obce nie zablokują usunięcia przełożonego.

8.4. Zastosowanie instrukcji CHECK

CHECK definiuje warunek logiczny obliczany dla pojedynczego rekordu. W dotychczasowym skrypcie znajduje się 16 nazwanych ograniczeń CHECK. Jeśli INSERT/UPDATE narusza warunek (wyrażenie zwraca FALSE), operacja jest odrzucana.

Tabela	Nazwa ograniczenia	Sprawdzany warunek i cel
Zone	ck_zone_capacity	capacity > 0 — Pojemność strefy musi być dodatnia.
Employee	ck_employee_not_own_manager	manager_id IS NULL OR manager_id <> employee_id — Zakaz wskazania siebie jako bezpośredniego przełożonego.
Contract	ck_contract_dates	end_date IS NULL OR end_date >= start_date — Umowa bezterminowa lub nieujemny okres obowiązywania.
Contract	ck_contract_fraction	employment_fraction > 0 AND <= 1 — Wymiar etatu w zakresie (0; 1].
Annex	ck_annex_number	annex_number > 0 — Dodatni numer aneksu.
Annex	ck_annex_fraction	employment_fraction > 0 AND <= 1 — Wymiar etatu po zmianie w zakresie (0; 1].
Annex	ck_annex_dates	end_date IS NULL OR end_date >= effective_from — Koniec obowiązywania aneksu nie może poprzedzać jego wejścia w życie.
Schedule	ck_schedule_dates	period_to >= period_from — Poprawna kolejność dat grafiku.
Schedule	ck_schedule_version	version_number > 0 — Dodatni numer wersji grafiku.
Schedule	ck_schedule_status	status IN ('draft', 'published', 'archived') — Tylko dozwolone statusy grafiku.
Shift	ck_shift_times	end_time > start_time — Zmiana musi kończyć się później, niż się zaczęła.
Availability	ck_availability_times	available_to > available_from — Prawidłowy, niepusty przedział dostępności.
Absence	ck_absence_dates	end_date >= start_date — Koniec nieobecności nie wcześniej niż początek.
Absence	ck_absence_status	status IN ('registered', 'cancelled') — Tylko zarejestrowana lub anulowana

Tabela	Nazwa ograniczenia	Sprawdzany warunek i cel
		nieobecność.
Registration	ck_registration_event_type	event_type IN (work_start, work_end, break_start, break_end) — Dozwolone typy zdarzenia rejestracji pracy.
Registration	ck_registration_verification	verification_status IN (unverified, valid, needs_explanation) — Dozwolone wyniki weryfikacji rejestracji.

```
CONSTRAINT "ck_zone_capacity" CHECK ("capacity" > 0),
CONSTRAINT "ck_contract_fraction" CHECK (
   "employment_fraction" > 0 AND "employment_fraction" <= 1
),
CONSTRAINT "ck_shift_times" CHECK ("end_time" > "start_time"),
CONSTRAINT "ck_schedule_status" CHECK (
   "status" IN ('draft', 'published', 'archived')
)
```

Przykład: INSERT do Zone z capacity = 0 zostanie odrzucony. Próba zapisania zmiany, której end_time nie jest późniejsze od start_time, również zakończy się błędem. Dzięki nazwom ck_* PostgreSQL wskazuje konkretny naruszony warunek.

Ważne: CHECK uznaje wyrażenie za spełnione także wtedy, gdy ma ono wynik UNKNOWN (NULL). Dlatego w kolumnach obowiązkowych — np. capacity, employment_fraction, start_time — zastosowano dodatkowo NOT NULL. Dla zakończenia umowy end_date dopuszczono NULL świadomie: oznacza brak wskazanej daty końcowej.

8.5. Ograniczenia CHECK a złożone reguły biznesowe

CHECK nie zastępuje walidacji wymagającej analizy innych wierszy lub tabel. W PostgreSQL typowy CHECK powinien odwoływać się tylko do wartości jednego aktualnie zapisywanego wiersza. Pozostałe reguły wymagają np. indeksów UNIQUE/EXCLUDE, funkcji z triggerami albo logiki aplikacyjnej.

Wymaganie projektu	Stan w SQL DDL / wymagane uzupełnienie
F12 — brak cykli przełożonych	CHECK blokuje jedynie pracownika jako własnego przełożonego; cykl A → B → A wymaga odrębnego sprawdzenia.
F20 — zgodność przydziałów	UNIQUE blokuje podwójny przydział do tej samej zmiany, ale nie sprawdza nakładania się różnych zmian, absencji, dostępności, ważności umowy i pojemności stref.
F21–F22 — publikacja i wersje	Częściowy indeks unikalny nie dopuszcza dwóch opublikowanych wersji dla identycznego działu i identycznych granic okresu; nie eliminuje okresów częściowo zachodzących.
F24 — deklaracje dostępności	CHECK pilnuje kolejności dat; brak zakazu nakładających się/sprzecznych deklaracji tej samej osoby.
F30–F32 — sekwencja rejestracji	CHECK ogranicza typ i status zdarzenia; nie weryfikuje par start/koniec ani czasu pracy po odjęciu przerw.
F36 — audyt zmian	Tabela Audit przechowuje strukturę danych audytu; automatyczne wyzwalacze zapisujące operacje nie są częścią tego etapu.

8.6. Implementacja i sprawdzenie więzów

Pliki SQL opracowano dla PostgreSQL 14+ i schematu work_time. Udostępniono dwie alternatywne ścieżki wdrożenia. Nie należy wykonywać pełnego CREATE TABLE ponownie na istniejącej bazie.

Stan bazy	Sposób postępowania
Nowa, pusta baza	Uruchomić baza_postgresql_z_integralnoscia.sql, a następnie 04_testy_wiezow_integralnosci.sql.
Baza utworzona wcześniej	Uruchomić tylko 03_migracja_wiezow_integralnosci.sql, a następnie 04_testy_wiezow_integralnosci.sql.

```
-- Wariant A: nowa baza
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 \
  -f baza_postgresql_z_integralnoscia.sql

-- Wariant B: istniejąca baza (ZAMIAST wariantu A)
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 \
  -f 03_migracja_wiezow_integralnosci.sql

-- Weryfikacja po A albo B
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 \
  -f 04_testy_wiezow_integralnosci.sql
```

Skrypt testów wypisuje katalog obowiązujących kluczy obcych wraz z akcjami ON DELETE/ON UPDATE i liczbę ograniczeń CHECK. Następnie sprawdza przykładowe naruszenia CHECK, UNIQUE i FOREIGN KEY, działanie CASCADE (usunięcie i aktualizacja), SET NULL i RESTRICT. Dane wstawiane w testach są wycofywane instrukcją ROLLBACK. Uwaga: wartości sekwencji IDENTITY mogą się zwiększyć także po wycofaniu transakcji.

Zakładany rezultat: 24 tabele, 16 nazwanych CHECK i 10 jawnie opisanych w rozdziale kluczy obcych z wybranymi akcjami. Pozostałe klucze obce działają z domyślnym NO ACTION. Nie przeprowadzono uruchomienia na rzeczywistej instancji PostgreSQL w środowisku przygotowania plików; dlatego komunikaty NOTICE OK są oczekiwanym wynikiem testów, a nie raportem z wykonanego wdrożenia.

8.7. Wnioski

Opracowane więzy integralności zabezpieczają podstawową spójność struktury danych i najważniejsze reguły dotyczące pojedynczych rekordów. Wybrane kaskady usprawniają zarządzanie relacjami wiele-do-wielu, bez usuwania historycznych danych o zatrudnieniu i czasie pracy. Wprowadzenie klucza głównego assignment_id nie osłabia unikalności przydziału, ponieważ zachowano ograniczenie UNIQUE(shift_id, employee_id). Pełne reguły kolizji harmonogramów, hierarchii służbowej, pojemności stref i audytu wymagają oddzielnej implementacji.

Źródła: „Bazy projekt-2.pdf", rozdz. 1–5; „bazy projekt (1).sql" i poprzednia wersja baza_postgresql.sql; PostgreSQL Documentation (Constraints): https://www.postgresql.org/docs/14/ddl-constraints.html.

Załączone pliki skryptów

- baza_postgresql_z_integralnoscia.sql — kompletne DDL dla nowej bazy, zawiera 24 tabele i wybrane jawne działania ON DELETE/ON UPDATE.
- 03_migracja_wiezow_integralnosci.sql — aktualizacja kluczy obcych w bazie utworzonej wcześniej.
- 04_testy_wiezow_integralnosci.sql — testy integralności z wycofaniem danych testowych.